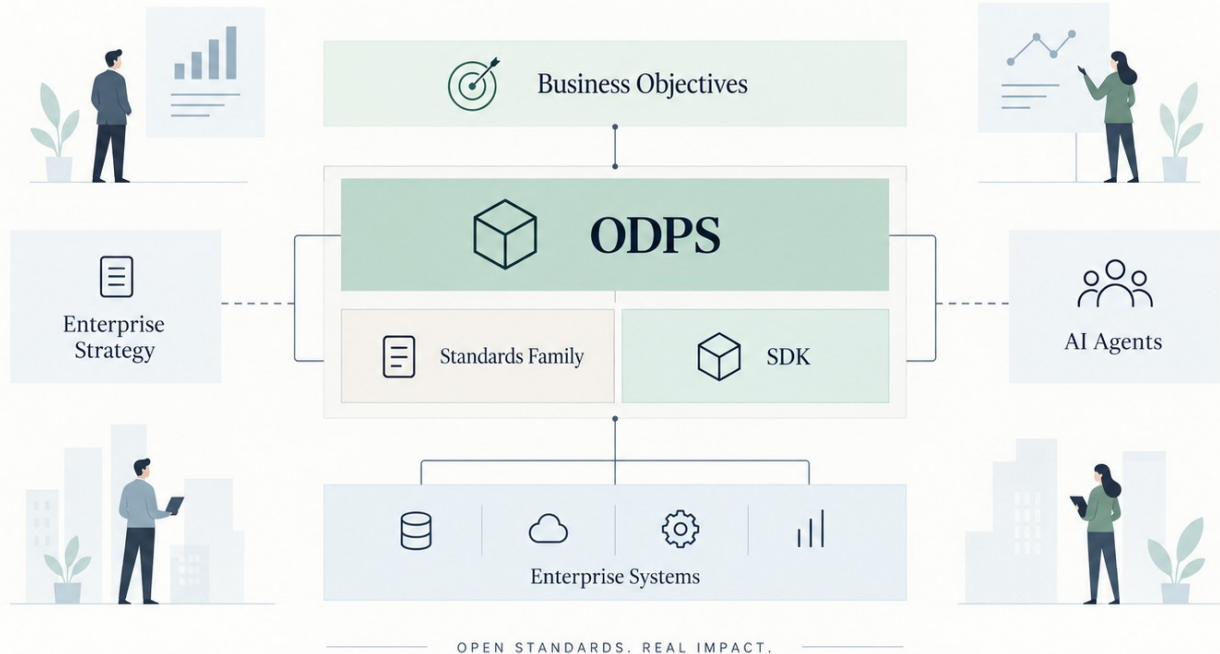


ODPS: From Data Products to Agent-Ready Data Ecosystems

The evolution, architecture and enterprise application of the Open Data Products standards family



Open standards. Real products. Agent-ready context.

Dr. Jarkko Moilanen

Igniter and Maintainer, Open Data Product Specification

Enterprise data product standards, architecture and AI-agent adoption

About this whitepaper

This whitepaper presents the evolution, architecture and enterprise application of the Open Data Product Specification (ODPS) and the wider Open Data Products standards family. It combines published specification material, public implementation evidence and the author's first-hand practitioner perspective as the igniter and maintainer of ODPS.

The paper is written for chief data officers, chief technology officers, heads of AI, enterprise architects, data product leaders, platform teams and governance leaders who need a practical way to connect product thinking, machine-readable metadata and agent-ready execution.

IMPORTANT DISTINCTION

"Open" describes the openness of the specification. It does not mean that the data described by an ODPS document is public or openly licensed. A data product represented with ODPS may be internal, restricted, commercial or public. [1][3]

Standards status in this paper is stated as of 16 September 2026. ODPS 4.1 is the current production release. ODPC, ODPG and ODPV are at version 1.0. ODPR is a draft workflow specification. [1][2][7][8][9][10]

This paper is not a normative specification. Where it recommends architecture, governance or an adoption sequence, those recommendations are presented as practitioner guidance rather than as requirements imposed by the standards.

Author perspective

I started the work that became ODPS from a business-first problem: organizations wanted to manage data as products, but the available metadata models mainly described technical assets. Over time the specification had to become portable, machine-readable, contract-aware, reusable, portfolio-aware and suitable for machine consumers. That progression explains why ODPS is now best understood as part of a standards family rather than as a single schema.

Contents

- Executive Summary
- 01 The Enterprise Data Product Problem
- 02 Why a Data Product Needs a Product Definition
- 03 From 2019 Idea to 2026 Standards Family
- 04 The Open Data Products Standards Family
- 05 From Standards to Working Practice: SDK, Knowledge and Skills
- 06 ODPS as the Product Control Layer
- 07 How ODPS Fits the Wider Standards Stack
- 08 Why AI Agents Change the Requirement
- 09 Enterprise Patterns and Public Implementation Signals
- 10 Enterprise Adoption Path
- 11 Governance, Risk and Operating Model
- 12 What Organizations Should Do Next
- About the Author
- References

Executive Summary

Enterprise data environments contain more metadata than ever, but that metadata is usually fragmented by tool and technical function. Catalogs describe assets. API managers describe endpoints. quality tools describe tests. IAM systems describe permissions. legal systems hold terms. strategy tools hold objectives and KPIs. The missing layer is a portable description of the data product as a product.

ODPS addresses that layer. It is a vendor-neutral, machine-readable product definition that brings together business purpose, ownership, access, quality, service expectations, licensing, commercial conditions, contracts and strategy. ODPS does not replace the systems that deliver or enforce these concerns. It creates a common product-level envelope that those systems and consumers can understand. [1][2]

The standard has matured in stages. ODPS 1.0 established a broad business, technical, legal and ethical model. The 2.x generation made the description more standalone and portable. The 3.x generation moved toward YAML-first operation, “as code” practices and data contract composition. ODPS 4.0 introduced reusable components and stronger modularity. ODPS 4.1 added productStrategy so product definitions can connect to objectives and KPIs. [3][4][5][6][1]

In 2026 the work expanded into a standards family. ODPC adds catalog and portfolio objects. ODPG adds relationship graphs. ODPV adds a shared vocabulary. ODPR, still draft, adds declarative workflow recipes. Together they create a progression from individual product description to portfolio context, semantic consistency, graph reasoning and repeatable workflow control. [2][7][8][9][10]

The implementation story is also maturing. The Open Data Products SDK provides a common Python and CLI surface for validation, explanation, generation, catalogs, graphs, portfolio workspaces, contracts and agent workflows. The project knowledge base adds applied examples, FAQs and SDK guidance, while five Masterclasses provide structured learning paths. These pieces matter because enterprise adoption fails when a standard is treated as documentation rather than as a working practice. [11][12]

AI agents increase the importance of this architecture. An API or MCP tool tells an agent what it can call. A product definition tells the agent what the resource is, why it exists, who owns it, whether it is fit for the task, what quality and service expectations apply, what its permitted use is, and how it relates to business intent. ODPC, ODPG, ODPV and ODPS together provide structured context before runtime authorization. MCP or another runtime protocol then handles interaction with tools and services. [8][9][11][21]

CORE PROPOSITION

The enterprise role of ODPS is not to replace catalogs, contracts, APIs, IAM, quality systems or observability. Its role is to connect them around a portable definition of the data product. In agentic systems, that product context becomes the layer between machine reasoning and controlled execution.

For most established enterprises, adoption should be incremental. Start with a representative set of products. Define a minimal enterprise profile. Validate manifests in CI. Reference existing contracts and interface specifications instead of duplicating them. Add ODPC when portfolio management is needed, ODPG when relationships have operational value, and ODPV early enough to control terminology. Expose products to agents first for read-only discovery, then add controlled execution behind IAM, policy and audit. Treat ODPR as an experimental workflow layer until it reaches a production release. [7][8][9][10][11]

The Enterprise Data Product Problem

THE INFORMATION EXISTS. THE PRODUCT CONTEXT DOES NOT.

A modern enterprise rarely lacks metadata. It lacks coherence. A single data product might be represented differently in a cloud catalog, an API gateway, a data contract, a quality platform, an access-control system, a marketplace and a strategy portfolio. Each representation serves its own system, yet no single portable artifact explains the product as a whole.

That fragmentation creates practical costs. Producers repeat descriptions across systems. Consumers infer purpose from technical names. governance teams reconcile ownership and access rules manually. Commercial teams maintain pricing and license details separately. Platform teams build mappings between proprietary metadata models. AI teams often give agents tool access without giving them structured product context.

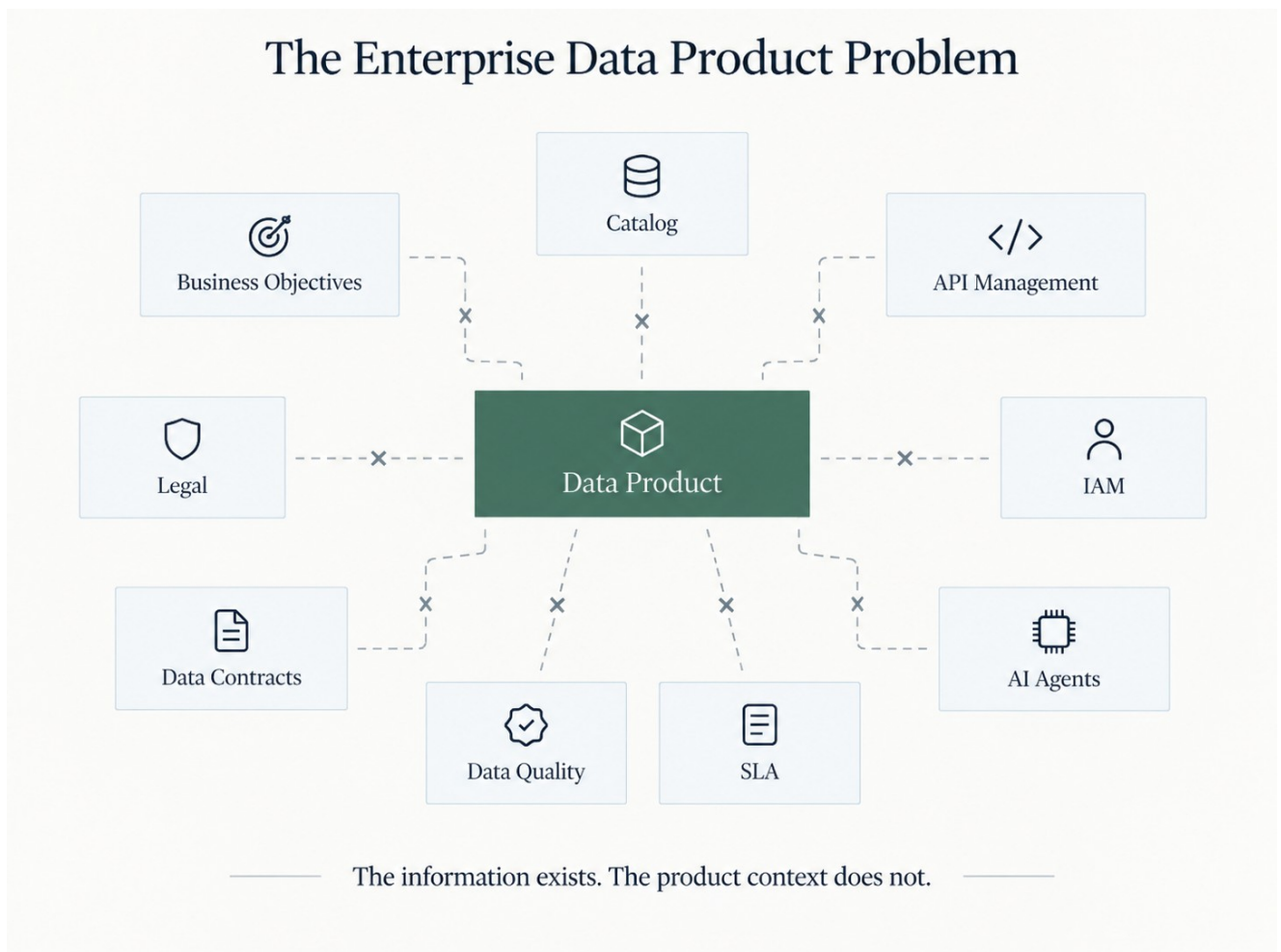


Figure 1. Enterprise product context is often distributed across systems that were designed for different concerns.

Dataset metadata is necessary, but incomplete

Technical metadata answers important questions: Where is the data? What columns exist? What is the schema? When was it updated? Those answers do not fully answer product questions: Who is accountable? Who is the intended consumer? What outcome does the product support? What service level should a consumer expect? What access method is approved? What contract, license or price applies?

This gap becomes visible when organizations try to operate data products at scale. A product operating model needs technical facts and product intent in the same lifecycle. ODPS emerged to make that product intent machine-readable and portable across systems. [1][3]

The recurring enterprise failure modes

Failure mode	What happens	Why it matters
Catalog lock-in	Product meaning is encoded in one vendor model or portal.	Moving or federating products requires custom mappings and manual reconciliation.
Purpose without structure	Business value is written in slides, tickets or tribal knowledge.	Consumers and agents cannot reliably determine fitness for use.
Ownership without accountability	An owner field exists, but commitments are not tied to service, quality or lifecycle.	Operational responsibility is unclear when products fail.
Disconnected contracts	Detailed data contracts exist outside the product description.	Consumers must discover the contract separately and interpret how it applies.
Declared quality without evidence	Quality targets are documented but not connected to observed runtime state.	A product may look trustworthy while current behavior differs.
Agent access without product context	An agent sees a callable endpoint but not purpose, restrictions or business relevance.	Tool availability is mistaken for approved product suitability.

DESIGN PRINCIPLE

Treat the data product as a first-class object that connects intent, expectations, access and operation. Do not force one catalog, API manager or governance tool to become the definition of the product.

Why a Data Product Needs a Product Definition

A PORTABLE MANIFEST FOR PURPOSE, TRUST, ACCESS AND VALUE.

ODPS is best understood as a portable product manifest. It describes what a data product is, why it exists, who owns and governs it, how it is accessed, what consumers should expect and how the product relates to wider business outcomes. [1]

The distinction between a data asset and a data product matters. A warehouse table, API, stream, file or model output is a delivery asset. A product definition adds the information required to manage that asset as something with consumers, value, lifecycle, service expectations, constraints and accountability.

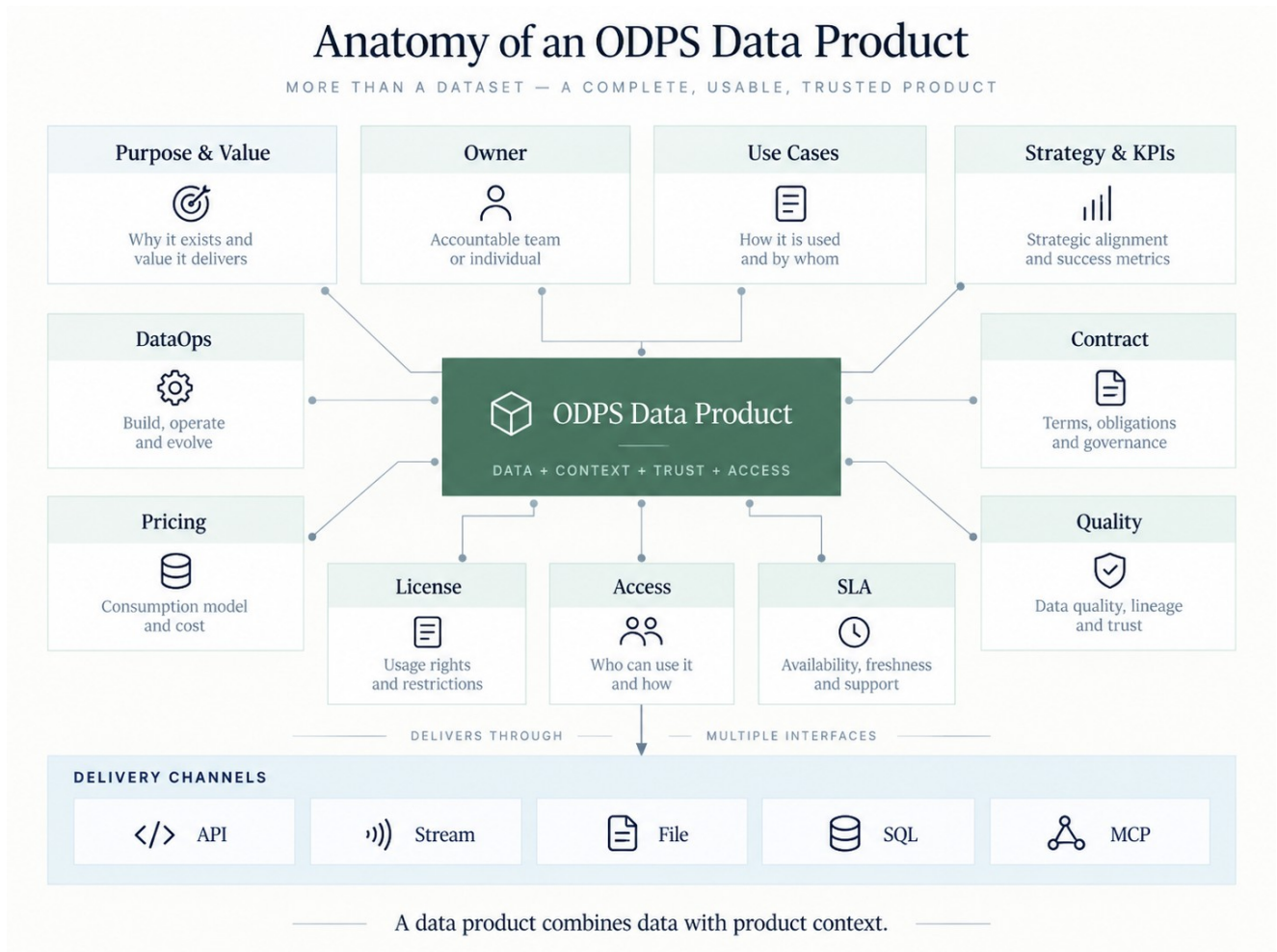


Figure 2. Conceptual anatomy of an ODPS data product. The illustration groups product-level concerns; runtime enforcement stays in enterprise systems.

A product definition does not create the runtime

An ODPS file does not store the data. It does not grant authorization. It does not execute quality checks or settle a payment. It does not operate an API gateway. It describes and references those concerns so that humans, platforms and machines have a common product-level view. This separation is a strength because the same product definition can sit above different technology stacks. [1]

The ODPS 4.1 functional areas

Area	What it expresses	Typical system relationship
Identity and details	Name, identifier, lifecycle, description, value proposition, use cases and presentation metadata.	Catalogs, marketplaces, product portals.
Product strategy	Objectives, product KPIs and relationships to higher-level business KPIs.	Portfolio and strategy management.
Contract	External or inline data contract references, including ODCS and DCS patterns.	Contract tooling and schema governance.
Quality and SLA	Declared quality expectations and service commitments.	Data quality, SRE and observability systems.
Access	Named access profiles, interface type, authentication method, format and documentation.	API gateways, warehouses, file delivery, MCP and other runtimes.
Licensing and pricing	Usage terms, rights, restrictions, pricing plans and payment references.	Legal, marketplace and commercial systems.
DataOps and operations	Operational metadata used to connect product definitions to delivery practices.	CI/CD, platform automation and support processes.

ODPS 4.0 introduced reusable references for areas such as SLA, quality, access and payment. ODPS 4.1 then added productStrategy, making business alignment a first-class part of the product manifest. [1]

A USEFUL MENTAL MODEL

The data itself is the delivered resource. The contract describes detailed expectations. The interface specification describes the technical interface. ODPS describes the product around those assets.

From 2019 Idea to 2026 Standards Family

THE GROWTH PATH REFLECTS RECURRING ENTERPRISE REQUIREMENTS.

The work that became ODPS started in 2019 alongside the Data Product Toolkit. The original problem was not a missing catalog schema. It was the absence of a common, reusable way to express business-oriented product concepts such as value, pricing and other metadata around data products.

An early specification effort began in 2019 and was later rewritten outside the original platform context. The open-standard incarnation began in late 2021, followed by the first public production release in February 2022. The distinction matters: ODPS has a 2019 conceptual origin and a later open-standard implementation history. [3]

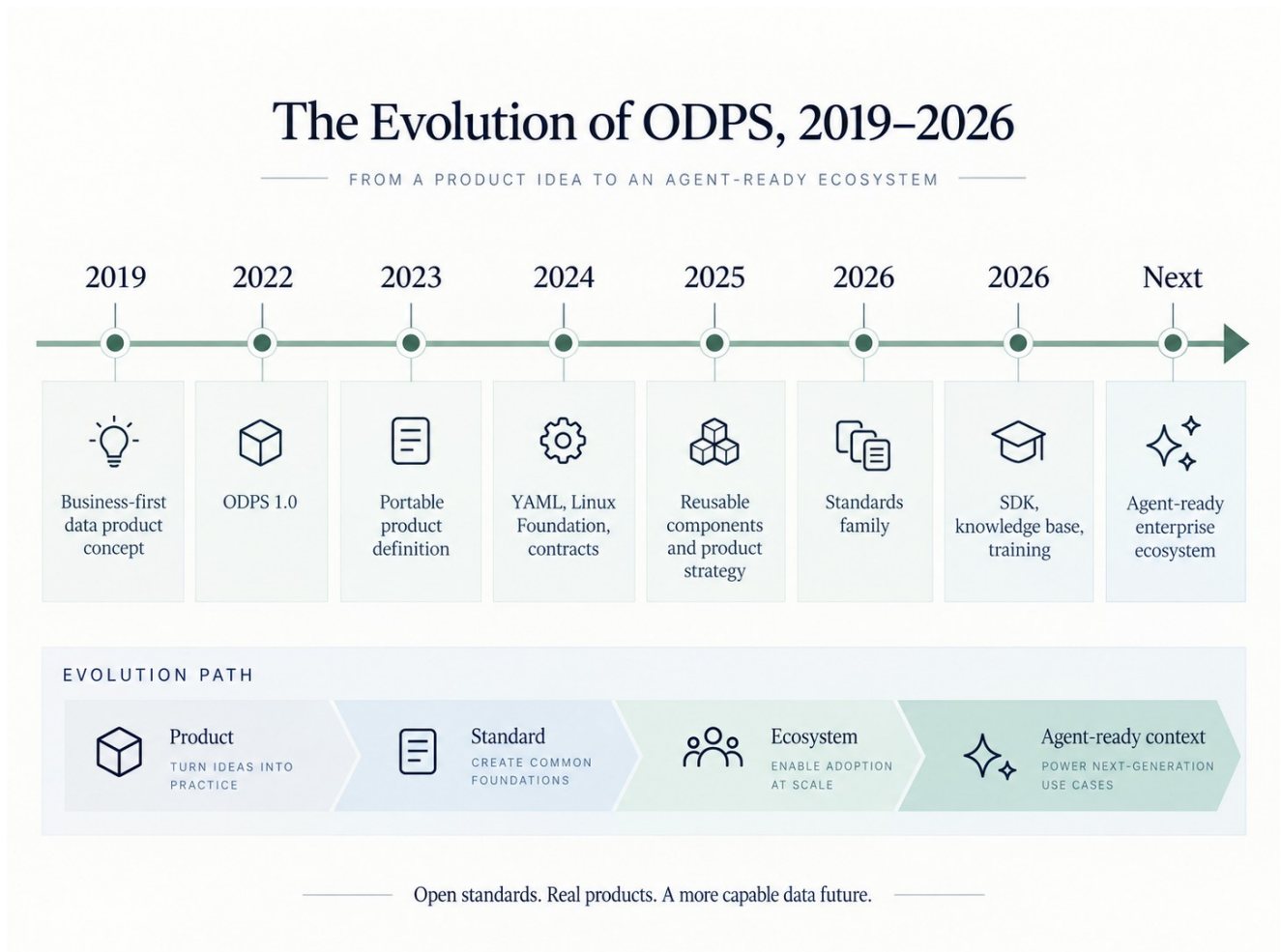


Figure 3. ODPS evolved from a business-first product idea into a wider implementation and agent context ecosystem.

Major releases and design shifts

Period	Milestone	What changed
2019-2021	Origins and independent rewrite	Business-first product metadata moved from an early toolkit/platform context toward a standalone open model.
Feb 2022	ODPS 1.0	First production release. Established the broad technical, business, legal and ethical product model. [3]
2023	ODPS 2.0 / 2.1	Shift toward a standalone product description, machine validation and practical JSON Schema examples. [4]
2024	ODPS 3.0 / 3.1	YAML-first operation, Linux Foundation hosting, “as code” practices and explicit data contract support. [5][6]
Jul 2025	ODPS 4.0	Reusable components and references strengthened modular enterprise use.
Oct 2025	ODPS 4.1	productStrategy connected products with objectives and KPIs. [1]
Jun 2026	ODPC / ODPG / ODPV 1.0	Portfolio, graph and vocabulary concerns became separate standards. [2][7][8][9]
2026	ODPR draft + maturing SDK	Workflow recipes and a cross-standard implementation surface moved the family closer to repeatable, agent-oriented operation. [10][11]

Why the family split matters

A single data product file should not become the container for every portfolio, graph, vocabulary and workflow concept. The 2026 family separates those responsibilities. That keeps ODPS focused on the individual product while allowing organizations to add wider context only when they need it.

The result is a two-dimensional maturity path. Horizontally, the family expands from product definition into portfolio, graph, vocabulary and workflow. Vertically, the ecosystem expands from specifications into schemas, examples, an SDK, a knowledge base and structured training. That combination is more useful for enterprise adoption than version numbers alone.

The Open Data Products Standards Family

SEPARATE CONCERNS, SHARED LANGUAGE, CONNECTED CONTEXT.

The current family contains five specifications. They should not be treated as five equal replacements for existing enterprise systems. Each has a narrow role that becomes more valuable when combined with the others. [2][7][8][9][10]

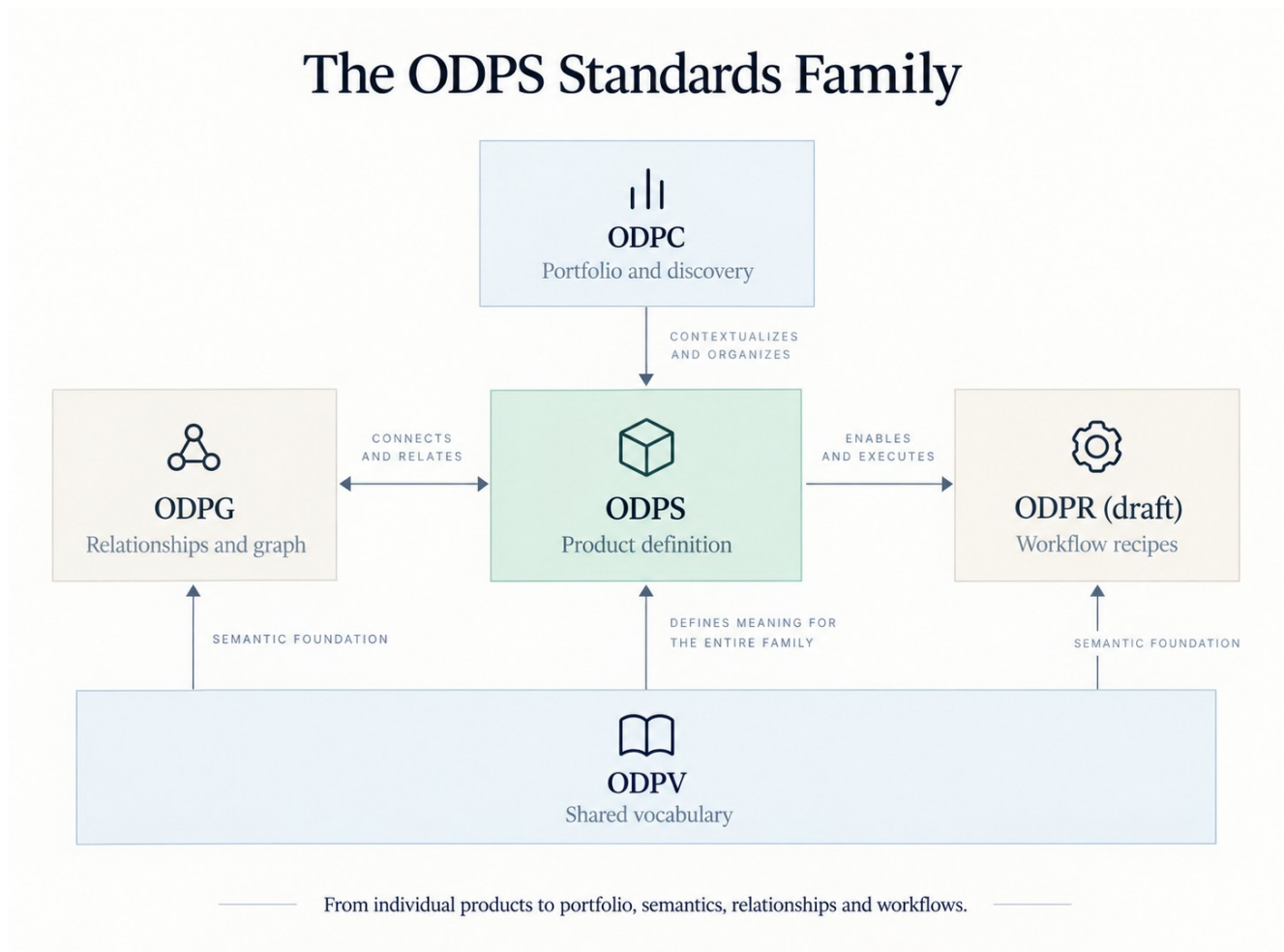


Figure 4. The standards family separates product, portfolio, relationships, vocabulary and workflow concerns. ODPR remains draft.

Standard	Primary question	Role
ODPS 4.1	What is this data product?	Defines one product: purpose, ownership, strategy, contract references, quality, SLA, access, licensing, pricing and operational metadata.
ODPC 1.0	What belongs in this portfolio?	Defines product references, use cases, business objectives, KPIs, signals, catalogs and catalog items for discovery and prioritization.
ODPG 1.0	How are these things related?	Defines graph relationships between products, objectives, use cases, KPIs, policies, APIs, workflows, agents and other assets.
ODPV 1.0	What do shared terms mean?	Provides controlled vocabulary and relationship language across the family, with machine-friendly serializations.
ODPR draft	What repeatable workflow should be followed?	Defines declarative recipes, provider profiles, inputs, outputs, checks, gates, context preferences and execution modes.

ODPC: portfolio and discovery

ODPC moves beyond isolated product descriptions. It models reusable portfolio objects such as ProductReference, UseCase, BusinessObjective, KPI, Signal, Catalog and CatalogItem. It is explicitly ODPS-native rather than ODPS-only, which allows references to products described through other enterprise or vendor models. [7]

ODPG: relationships and reasoning

ODPG is the relationship layer. Its value appears when organizations need dependency analysis, business-value mapping, governance propagation, impact analysis or graph-based context for AI. A catalog finds candidate assets. A graph explains how those assets connect to objectives, consumers, policies, workflows and other products. [8]

ODPV: semantic consistency

ODPV provides the controlled terminology that keeps the family aligned. Its published forms include YAML, JSON, JSON-LD, SKOS Turtle and JSONL. The project also includes cross-specification drift checks and agent-oriented vocabulary helpers. [9]

ODPR: workflow contracts

ODPR is deliberately different from a script. The draft describes repeatable workflow contracts: steps, inputs, outputs, checks, gates, provider profiles and execution expectations. It is intended to remain independent from a single SDK, CI system, model provider or orchestration engine. It should be treated as experimental until a production release exists. [10]

FAMILY LOGIC

ODPS defines the product. ODPC organizes the portfolio. ODPG connects the relationships. ODPV stabilizes shared meaning. ODPR describes repeatable workflows.

From Standards to Working Practice

SDK, KNOWLEDGE, EXAMPLES AND STRUCTURED LEARNING.

A specification gains enterprise value when teams can validate it, automate it, teach it and integrate it. The Open Data Products ecosystem now includes a maturing Python SDK, a practical knowledge base and five Masterclasses that cover standards, SDK workflows, monetization, product thinking and governance. [11][12]

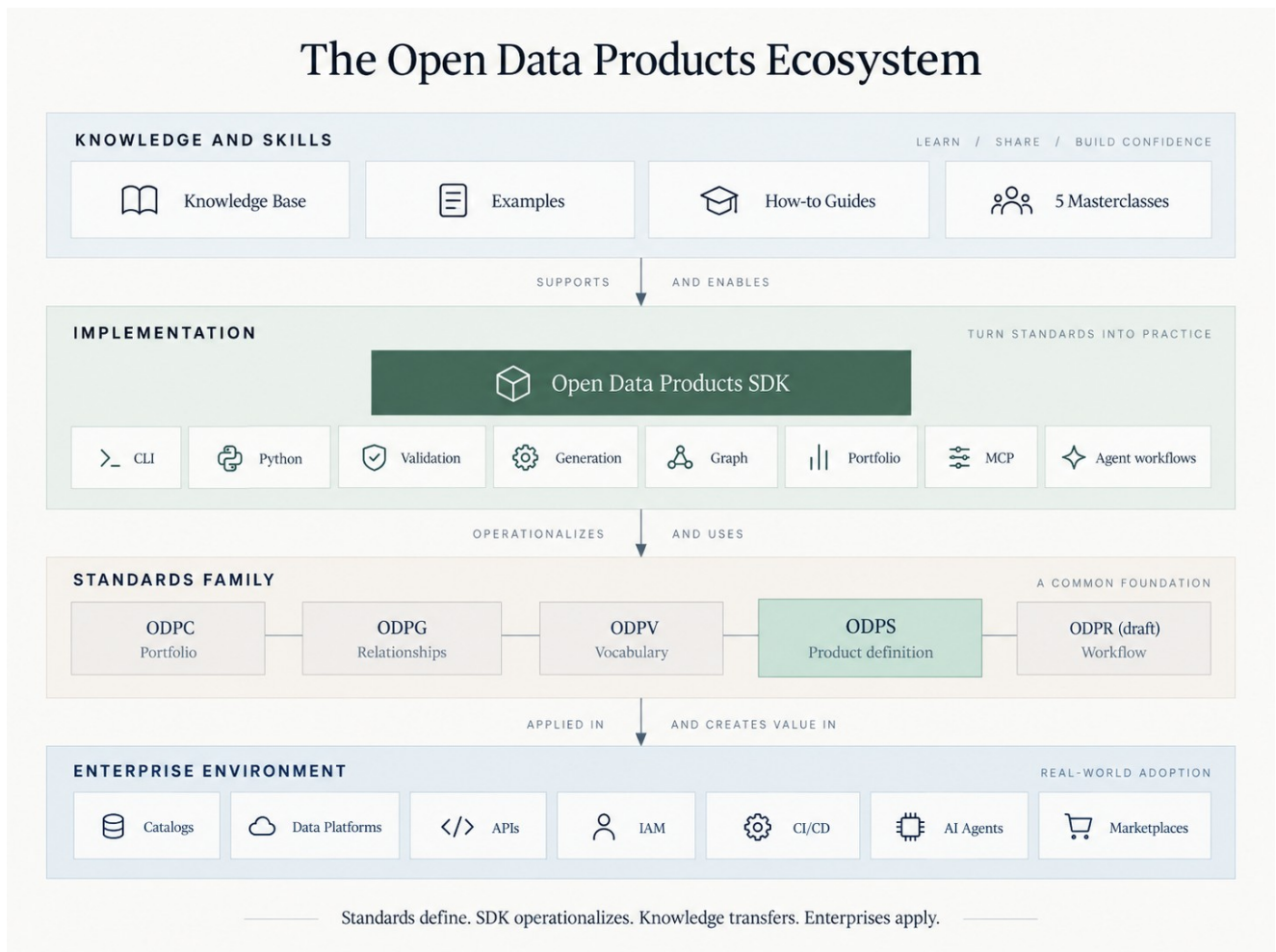


Figure 5. The wider ecosystem links standards, implementation tooling, learning assets and enterprise use.

The Open Data Products SDK

The SDK provides a consistent programmatic surface across the standards family. The current guide describes support for loading, detecting, validating, explaining, searching, traversing and summarizing ODPS, ODP, ODPG, ODPV and ODPR artifacts. The unified CLI includes validation, generation, catalog, graph, vocabulary, portfolio, contract, resource, manifest and MCP workflows. [11]

The agent surface is important. The SDK can expose a local MCP server, produce an agent manifest, traverse ODPG graphs, generate compact context artifacts and work with ODPS contract references. This moves agent-readiness from a conceptual claim toward an implementable pattern. [11]

Knowledge base and applied examples

The knowledge base complements formal specification text with FAQs, reusable YAML, SDK guides and practical resources. As of September 2026 the repository reports 34 FAQs, 14 examples, 19 SDK guides, three resource areas and five courses. [12]

That applied layer is essential for adoption. Standards describe what a compliant representation means. Examples and SDK workflows show how teams actually produce, validate and integrate those representations in day-to-day work.

Five Masterclasses as a capability layer

Structured training reduces a common failure mode in standards programs: a small architecture group understands the specification, while product owners and delivery teams continue using older practices. The current learning portfolio spans ODPS, agent-ready SDK workflows, monetization, data product thinking and minimum lovable governance. [12]

MATURITY IS MORE THAN A SCHEMA VERSION

A usable standards ecosystem needs a specification, validation, implementation tooling, examples, learning paths and repeatable operating practices. The SDK, knowledge base and training layer are therefore part of the adoption story, even though they are not themselves standards.

06

ODPS as the Product Control Layer

DESCRIBE AND CONNECT. LET RUNTIME SYSTEMS ENFORCE AND OPERATE.

ODPS fits most naturally above existing enterprise infrastructure. It should not become a replacement program for the catalog, data platform, IAM system, API gateway, observability stack or contract tooling. Instead, it provides a product-level control layer that references and connects those systems around a portable definition. [1]



Figure 6. Conceptual control-layer view. Read the highlighted ODPS layer as product definition and product commitments; specialized systems remain systems of record for runtime behavior.

Four layers to keep separate

Layer	System of record	Examples
Product intent	ODPS, ODPC, ODPG, ODPV	Purpose, ownership, lifecycle, strategy, use cases, relationships and catalog context.
Interface and contract definition	Specialized standards	ODCS for data contracts, OpenAPI for HTTP interfaces, AsyncAPI for event-driven interfaces.
Runtime enforcement	Enterprise platforms	IAM, policy engines, gateways, warehouses, data quality engines, orchestration and service controls.
Runtime evidence	Observability and lineage	Measured quality, SLO results, audit events, OpenLineage or equivalent runtime evidence.

Declared state versus observed state

A product may declare a freshness target, uptime commitment or quality threshold. That declaration is not proof that the product currently meets the target. Mature architectures compare declared state with observed state. The product manifest expresses the commitment. Runtime monitoring and lineage systems provide evidence.

This separation is especially important for AI. An agent should be allowed to reason over approved product metadata, but it should not interpret a declaration as authority to access the resource or as proof of current compliance. Authorization and high-consequence controls stay outside the model.

References are dependencies

ODPS 4.x supports reusable internal and external references. That reduces duplication but introduces supply-chain concerns. Enterprises should govern reference origins, pin versions, cache approved artifacts where appropriate and monitor broken or changed dependencies. Product metadata should be treated with the same change discipline as other machine-readable operational artifacts.

ARCHITECTURE RULE

ODPS should describe how a product is meant to be consumed. IAM and policy decide whether the current identity is allowed to consume it.

07

How ODPS Fits the Wider Standards Stack

COMPOSE SPECIALIZED STANDARDS INSTEAD OF REBUILDING THEM.

ODPS is most useful as an integration standard. It should reference specialized standards that already model narrower technical concerns with greater depth. This keeps the product definition broad without making it monolithic.

Standard	Primary scope	Relationship to OpenDataProducts ODPS
ODCS	Data contract structure, semantics, quality and service expectations.	Complementary. ODPS can reference or embed contract information while retaining wider product context.
OpenAPI	Machine-readable HTTP API description.	Complementary. OpenAPI describes the interface; ODPS describes the product around the interface. [17]
AsyncAPI	Machine-readable event and message-driven API description.	Complementary for stream or event output ports. [18]
DCAT 3	RDF vocabulary for interoperable web data catalogs.	Adjacent. DCAT is strong for federated dataset/service cataloging; ODPS adds richer product, strategy, service and commercial context. [16]
OpenSLO	Vendor-neutral service-level objectives.	Complementary where declared ODPS service expectations need operational SLO artifacts. [19]
OpenLineage	Runtime lineage events for datasets, jobs and runs.	Complementary runtime evidence rather than product definition. [20]
MCP	Agent-to-tool/data runtime protocol.	Complementary. ODPS describes an approved product and access option; MCP handles runtime interaction and authorization patterns. [21]
Bitol ODPS	Separate Open Data Product Standard in the Bitol ecosystem.	Alternative product descriptor with an acronym collision. Namespace the project and version explicitly. [22]

DCAT is adjacent, not identical

DCAT 3 is a W3C Recommendation designed to facilitate interoperability between web data catalogs. Its core focus is cataloging datasets and data services using RDF and supporting discovery across catalogs. ODPS places greater emphasis on product management concepts such as value proposition, access, service commitments, pricing, licensing and strategy. An organization can therefore expose DCAT for public or federated catalog interoperability while maintaining richer ODPS manifests for product management. [16]

A naming issue that enterprises should handle explicitly

A separate Linux Foundation AI & Data ecosystem, Bitol, also uses the acronym ODPS for its Open Data Product Standard. The Bitol repository identifies its current public release as version 1.0.0. This creates a real namespace collision. Architecture documents, schema registries, procurement requirements and APIs should use explicit identifiers such as “OpenDataProducts ODPS 4.1” and “Bitol ODPS 1.0.0”. [22]

COMPOSITION-FIRST APPROACH

Use ODPS as the product envelope. Keep detailed interface, contract, SLO, lineage and runtime semantics in the standards and systems designed for those concerns.

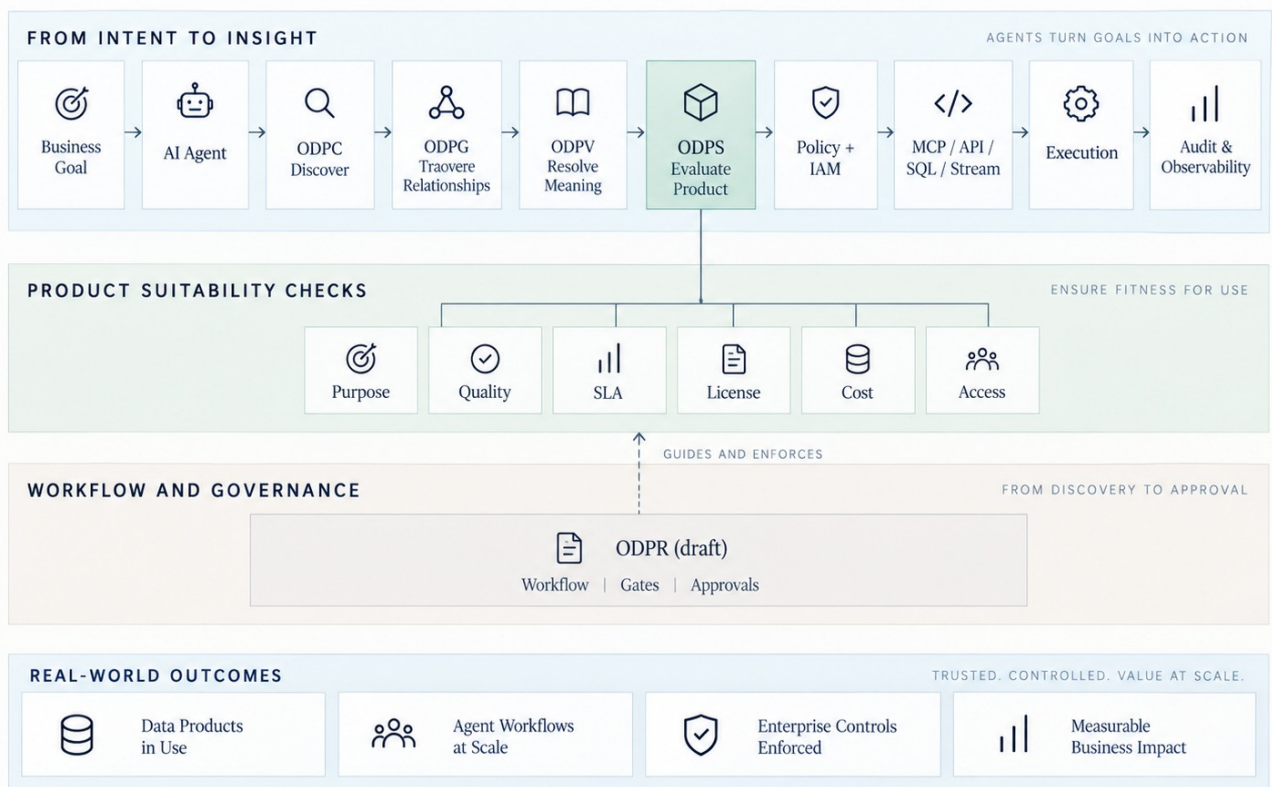
Why AI Agents Change the Requirement

TOOL ACCESS IS NOT THE SAME AS TRUSTED PRODUCT ACCESS.

Traditional application integration starts from interfaces. Agentic integration needs to start one level earlier. Before an agent calls a tool, it needs enough structured context to decide whether the underlying product is relevant, approved, trustworthy and fit for the task.

An API schema tells an agent which operations exist. It does not necessarily tell the agent why the data product exists, which objective it supports, which consumer it was designed for, what quality and freshness expectations apply, whether a license restricts use, whether cost is involved, or which downstream products depend on it.

Agent-Ready Data Product Architecture



Agents reason over product context. Enterprise controls retain authority.

Figure 7. Agent-ready architecture separates discovery and reasoning from policy, authorization and controlled execution.

A practical agent reasoning sequence

- Use ODPC to discover candidate products, use cases, objectives and portfolio objects.
- Use ODPG to traverse dependencies and business relationships.
- Use ODPV to resolve consistent terminology and relationship meaning.
- Inspect ODPS to evaluate purpose, ownership, contract, quality, SLA, access, license and cost.
- Pass the result through deterministic policy and IAM.
- Invoke the approved API, stream, SQL interface, file channel or MCP tool.
- Record audit, lineage and observed product-health evidence.

Reasoning is not authority

The language model should not become the enforcement point. The agent can interpret product context, compare alternatives and plan actions. Deterministic enterprise systems should decide authorization, spending limits, contractual restrictions, privileged writes and other high-impact transitions.

This distinction aligns with the direction of MCP authorization. The July 2026 MCP specification strengthened authorization hardening, including issuer validation and other controls. The architecture implication is straightforward: ODPS describes product context and access options; the runtime protocol and enterprise security systems enforce the actual permission boundary. [21]

Four useful agent patterns

Pattern	What the agent does	Control point
Discovery	Finds products aligned with a business goal and explains why they fit.	Read-only catalog and graph access.
Product onboarding	Drafts ODPS, catalog and graph artifacts from business and technical inputs.	Human approval of purpose, ownership, legal and access metadata.
Portfolio analysis	Finds gaps, overlaps, unsupported objectives and downstream impacts.	Evidence-based graph traversal and transparent reasoning.
Controlled execution	Selects a product and invokes an approved runtime interface.	Deterministic IAM, policy, approval and audit before action.

AGENT-READY PRINCIPLE

Agents reason over product context. Enterprise controls retain authority.

09

Enterprise Patterns and Public Implementation Signals

PORTABILITY, CATALOG AUTOMATION AND MACHINE-READABLE PRODUCT CONTEXT.

Public evidence shows several distinct ODPS adoption patterns. These examples are useful because they demonstrate different integration points rather than one prescribed platform architecture.

X-Road: enrich an existing technical interface

The Nordic Institute for Interoperability Solutions described a proof of concept in which ODPS metadata was embedded in X-Road OpenAPI service descriptions. The approach made data-product metadata available through an existing service discovery mechanism without requiring code or configuration changes to the X-Road Security Server. NIIS identified reduced manual catalog metadata, improved consistency, machine readability and easier maintenance as benefits. [13]

The architectural lesson is broader than X-Road. A technical interface definition can remain the source of interface semantics while ODPS adds product context around it. This is a clean example of composition rather than replacement.

Alation: AI-assisted product creation and portability

Alation announced its Data Products Builder Agent in May 2025 and stated that its data product definitions build on the Open Data Products Specification. The company positioned ODPS-based YAML as a way to keep product metadata open, portable, extensible, governed and machine-readable across platforms. [14]

This pattern is important for AI adoption. The agent is not only consuming existing data products. It is helping create, document and publish them while keeping the resulting definition portable rather than trapped inside one AI workflow.

FIWARE: interoperable data ecosystems

FIWARE Foundation and ODPS announced collaboration around interoperable data ecosystems, including integration of ODPS-oriented product definitions with FIWARE’s Business API Ecosystem. The pattern is relevant for smart cities, public sector, industrial and cross-organization data spaces where providers need a common product layer above heterogeneous infrastructure. [15]

Enterprise adoption signals

The ODPS project and the author's consulting site also publish adoption signals from enterprises such as BASF. These are useful indicators of practical interest, but they should be interpreted according to the evidence available. A testimonial or public reference is not the same as a full technical case study with independently measured outcomes.

EVIDENCE DISCIPLINE

Separate public implementation evidence, attributed testimonials and illustrative architecture patterns. A credible standards whitepaper should not turn all three into the same kind of claim.

10

Enterprise Adoption Path

START SMALL, PROVE PORTABILITY, THEN ADD CONTEXT AND AUTOMATION.

The safest enterprise pattern is additive. Do not start by replacing the catalog, IAM system, contract standard, lineage platform or API definitions. Start by creating a portable product representation above them and prove that the representation improves discovery, governance and interoperability.

Enterprise ODPS Adoption Path

FROM FOUNDATION TO ENTERPRISE VALUE



Adopt incrementally. Scale with governance.

Figure 8. A staged adoption path from assessment to federated scale.

Stage 1: assess the current product estate

Select a small set of representative products. Include more than one delivery pattern, for example a warehouse product, an API-backed product and a streamed product. Inventory current metadata, owners, contracts, quality expectations, access mechanisms, business objectives and known consumers.

Stage 2: define an enterprise ODPS profile

Do not require every optional field from day one. Define a practical minimum profile for the organization: identity, owner, lifecycle, value proposition, access, contract reference where relevant, core quality and service commitments, and strategy linkage where measurable objectives already exist. Register enterprise extensions and naming rules.

Stage 3: pilot and validate

Represent the selected products in ODPS 4.1. Store the manifests in version control. Validate schema and enterprise policy in CI. Use the SDK to detect, validate, explain and inspect artifacts. Treat every change as a reviewed product change rather than as an ungoverned metadata edit. [11]

Stage 4: integrate specialized standards and systems

Link existing data contracts, OpenAPI or AsyncAPI documents, SLO artifacts and other technical definitions rather than copying them into ODPS. Build adapters into the catalog, marketplace or product portal used by the organization.

Stage 5: add portfolio, vocabulary and graph context

Introduce ODPC when the organization needs consistent portfolio objects and business-demand connections. Introduce ODPV before enterprise extensions and domain terminology proliferate. Add ODPG where relationship traversal has a clear use case, such as impact analysis, objective coverage, dependency discovery or agent grounding. [7][8][9]

Stage 6: expose products to agents

Start with read-only discovery and explanation. Only after IAM, policy, audit and approval controls are mature should agents move toward controlled execution. The SDK's MCP surface provides one implementation path, but the underlying rule remains protocol-independent: discovery and reasoning do not grant authority. [11][21]

Stage 7: experiment with ODPR

Use ODPR for non-critical workflow recipes, CI checks, review packages or controlled agent workflows behind adapters. Pin the draft version and avoid hard production dependencies until the standard reaches production status. [10]

ADOPTION OBJECTIVE

The first success metric is not "how many YAML files exist." It is whether a product definition becomes portable, reviewable, connected to business intent and useful across more than one system or consumer.

11

Governance, Risk and Operating Model

A MACHINE-READABLE STANDARD STILL NEEDS HUMAN ACCOUNTABILITY AND RUNTIME CONTROLS.

The hardest part of ODPS adoption is rarely syntax. The difficult work is making ownership, value, quality expectations, service commitments, permitted use and product lifecycle explicit enough that they can be reviewed and governed.

Four governance layers

Layer	Accountability
Standards governance	Approved family versions, enterprise profile, extension registry, naming, mappings, upgrade policy and exceptions.
Domain product governance	Purpose, value proposition, lifecycle, users, quality promises, strategy, support expectations and metadata accuracy.
Platform control	Validation, CI gates, reference resolution, IAM integration, catalog ingestion, secrets handling, lineage and observability.
AI governance	Agent identities, tool allowlists, approval thresholds, provider policies, context handling, evaluation, audit and high-impact action controls.

Key implementation risks

Risk	Why it matters	Mitigation
Acronym collision	OpenDataProducts and Bitol both use ODPS.	Namespace project and version in schemas, catalogs and architecture documents.
Family maturity	ODPC, ODPG and ODPV reached 1.0 in 2026; ODPR is draft.	Pin versions, use adapters and avoid draft hard dependencies.
Extension fragmentation	Uncontrolled local fields recreate semantic inconsistency.	Maintain an extension registry and map to ODPV or established vocabularies.
Reference supply-chain risk	Shared external references affect many products.	Allowlist sources, pin versions, validate and cache approved artifacts.
False compliance assurance	Declared license, SLA or quality metadata can be mistaken for proof.	Keep legal review and runtime measurement outside the manifest.
Metadata leakage	Endpoints, product existence or commercial terms may be sensitive.	Classify and authorize metadata views as well as underlying data.
Agent privilege escalation	Discovery of an access path can be confused with permission to use it.	Separate identities, least privilege, deterministic policy and approval.
Prompt or metadata injection	Free text and linked documentation enter LLM context.	Treat retrieved metadata as data, constrain tools and isolate untrusted text.
Declared-versus-observed drift	Product commitments may diverge from runtime reality.	Reconcile manifests against quality, SLO, audit and lineage evidence.

Govern extensions early

Enterprise-specific metadata is expected. The goal is not to prohibit extensions but to prevent every domain from inventing a different term for the same concept. Each extension should have an owner, definition, data type, allowed values, lifecycle and mapping to established vocabulary where possible.

Do not put secrets in product or workflow metadata

ODPR explicitly separates provider and credentials references from raw secrets. The same principle should apply to ODPS access metadata. A machine-readable product definition should expose enough information to understand the access path without becoming a repository for tokens, passwords or API keys. [10]

GOVERNANCE OUTCOME

The manifest is the approved declaration of the product. Runtime evidence proves whether the product is behaving as declared.

What Organizations Should Do Next

USE A DECISION FRAMEWORK BEFORE COMMITTING TO ENTERPRISE ROLLOUT.

ODPS is most valuable when an organization has already outgrown dataset-centric metadata and needs product definitions to move across teams, tools and machine consumers. The right starting point is a focused assessment, not a large-scale metadata conversion program.

Readiness questions

Question	If the answer is “no”
Do we have a consistent definition of a data product across domains?	Start with product model alignment and a minimal enterprise profile.
Can the definition move between tools without losing meaning?	Prioritize portable ODPS representations and explicit mappings.
Are products linked to measurable business objectives or use cases?	Add productStrategy and, when needed, ODPC portfolio objects.
Are access, contract, quality and SLA expectations connected to the product?	Reference the relevant specialized artifacts from the product definition.
Do we understand product dependencies and downstream impact?	Add ODPG selectively where graph relationships have operational value.
Do humans and AI agents use the same approved product context?	Expose validated product metadata and shared vocabulary through governed discovery.
Does runtime evidence match declared commitments?	Connect quality, SLO, audit and lineage observations back to product health.
Are agent actions separated from model reasoning?	Put IAM, policy, approvals and audit between product selection and execution.

A pragmatic first engagement

For an enterprise evaluating ODPS, the first useful unit of work is a small portfolio slice: three to ten representative products, their current metadata and interfaces, and one or two business use cases. The output should be a gap assessment, an enterprise ODPS profile, mapped example products, an integration architecture and a recommendation on whether ODPC, ODPG, ODPV or agent workflows add immediate value.

That approach makes the decision reversible. The organization gets a concrete view of fit before changing its operating model or tooling. Because ODPS is a portable specification, the pilot artifacts remain useful even if the wider program stops or changes direction.

Where the standards family is heading

The growth path suggests a clear direction: data product definitions are becoming more machine-readable, more connected to business intent and more usable by automated systems. The key architectural challenge is to preserve separation of concerns while adding richer context. Product metadata should remain portable. specialized standards should keep their depth. Runtime security should remain deterministic. Agent workflows should remain inspectable.

STRATEGIC RECOMMENDATION

Adopt ODPS first as a governed interoperability layer. Prove portability and product accountability. Add portfolio, graph and agent capabilities only where they solve a real business or operating problem.

Closing Perspective

ODPS started from a practical question: if data is managed as a product, where is the machine-readable definition of that product? The answer has expanded over time because enterprise data products are not isolated objects. They exist in portfolios, depend on other assets, carry shared semantics, move through workflows and increasingly serve machine consumers as well as people.

The standards family now reflects that wider reality. ODPS provides the product definition. ODPC provides portfolio structure. ODPG provides relationships. ODPV provides shared meaning. ODPR is emerging as a workflow contract. The SDK gives developers and agents a practical implementation surface. The knowledge base and Masterclasses help teams move from specification text to working practice.

The AI-agent context makes this work more urgent, but it does not change the underlying discipline. An agent still needs the same things a good human consumer needs: purpose, ownership, quality, service expectations, permitted use and a clear path to access. What changes is that these concepts must be structured enough for machines to interpret consistently and controlled enough that interpretation does not become authority.

For enterprise architects, the practical value of ODPS is therefore not another metadata schema. It is a way to make the data product itself portable across the systems that describe, govern, deliver and consume it.

About the Author

Dr. Jarkko Moilanen is the igniter and maintainer of the Open Data Product Specification and a practitioner working at the intersection of data products, open standards, governance, enterprise architecture and AI-agent adoption.



His work on ODPS began from business-oriented data product design and evolved into an open standards family maintained under LF AI & Data. He has worked with public-sector, enterprise and technology organizations on data product operating models, metadata interoperability, portfolio management and AI-ready data practices.

This whitepaper reflects his practitioner perspective on how the standards fit together and how organizations can apply them incrementally. The standards remain open. Consulting work focuses on evaluating fit, defining enterprise profiles, designing integration architecture, supporting adoption and making product context usable by both people and AI systems.

Professional focus

- ODPS readiness and enterprise adoption
- Data product architecture and governance
- Open standards interoperability and mapping
- AI-agent-ready data product context
- Portfolio, graph and business-value alignment
- SDK-supported implementation and automation

jarkkomoilanen.com/services/odps/

opendataproductions.org

References

- [1] Open Data Product Specification 4.1, Linux Foundation project. <https://opendataproductions.org/v4.1/>
- [2] Open Data Products Standards Family, project resources and release status. <https://opendataproductions.org/>
- [3] Open Data Product Specification 1.0, first production release, 4 February 2022. <https://github.com/Open-Data-Product-Initiative/v1.0>
- [4] Open Data Product Specification 2.1, production release, 22 August 2023. <https://opendataproductions.org/v2.1/>
- [5] Open Data Product Specification 3.0, Linux Foundation project. <https://opendataproductions.org/v3.0/>
- [6] Open Data Product Specification 3.1, production release with data contract support. <https://opendataproductions.org/v3.1/>
- [7] Open Data Product Catalogs 1.0 (ODPC). <https://github.com/Open-Data-Product-Initiative/odpc-v1.0>
- [8] Open Data Product Graphs 1.0 (ODPG). <https://github.com/Open-Data-Product-Initiative/odpg-v1.0>
- [9] Open Data Product Vocabulary 1.0 (ODPV). <https://github.com/Open-Data-Product-Initiative/odpv-v1.0>
- [10] Data Product Recipe Specification (ODPR), draft 1.0 repository. <https://github.com/Open-Data-Product-Initiative/odpr-v1.0>
- [11] Open Data Products SDK, full user guide. <https://github.com/Open-Data-Product-Initiative/odp-agent-sdk/blob/main/docs/user/full-sdk-guide.md>
- [12] Open Data Product Standards Knowledge Base. <https://github.com/Open-Data-Product-Initiative/odps-knowledge-base>
- [13] Nordic Institute for Interoperability Solutions, "Unveiling the Open Data Product Specification", X-Road proof of concept. <https://www.niis.org/blog/2024/1/21/unveiling-the-open-data-product-specification>
- [14] Alation, "Alation Launches Data Products Builder Agent to Power AI-Ready Data", 12 May 2025. <https://www.alation.com/news-and-press/alation-launches-data-products-builder-agent-to-power-ai-ready-data/>
- [15] FIWARE Foundation and ODPS, interoperable data ecosystems collaboration. <https://fiware.org/news/fiware-foundation-and-odps-join-forces-to-drive-next-generation-interoperable-data-ecosystems/>
- [16] W3C, Data Catalog Vocabulary (DCAT) Version 3, Recommendation, 22 August 2024. <https://www.w3.org/TR/vocab-dcat-3/>
- [17] OpenAPI Specification. <https://spec.openapis.org/oas/>
- [18] AsyncAPI Specification and documentation. <https://www.asyncapi.com/docs/reference>
- [19] OpenSLO, open specification for service-level objectives. <https://github.com/OpenSLO/OpenSLO>
- [20] OpenLineage specification and object model. <https://openlineage.io/docs/spec/object-model/>
- [21] Model Context Protocol, 2026-07-28 specification release and authorization hardening. <https://blog.modelcontextprotocol.io/posts/2026-07-28/>
- [22] Bitol, Open Data Product Standard (ODPS) v1.0.0. <https://github.com/bitol-io/open-data-product-standard>